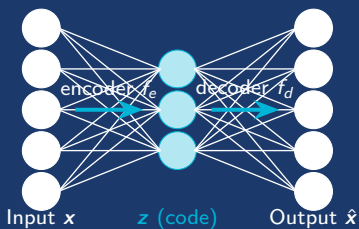


Autoencoders

Nonlinear Dimensionality Reduction
with Neural Networks



PCA as a Linear Autoencoder

Recall that PCA learns:

- **Encoder:** $\mathbf{z} = \mathbf{W}_1 \mathbf{x}$ ($L \times D$ matrix)
- **Decoder:** $\hat{\mathbf{x}} = \mathbf{W}_2 \mathbf{z}$ ($D \times L$ matrix)
- **Reconstruction:** $r(\mathbf{x}) = \mathbf{W}_2 \mathbf{W}_1 \mathbf{x} = \mathbf{W} \mathbf{x}$

Trained to minimize $\mathcal{L} = \sum_n \|\mathbf{x}_n - \mathbf{W} \mathbf{x}_n\|^2$.

Nonlinear Autoencoders

Replace linear maps with **nonlinear neural networks**:

$$r(\mathbf{x}) = f_d(f_e(\mathbf{x}; \theta_e); \theta_d)$$

$$\mathcal{L}(\theta) = \|r(\mathbf{x}) - \mathbf{x}\|^2 \quad \text{or} \quad -\log p(\mathbf{x}|r(\mathbf{x}))$$

Theorem [Japkowicz et al., 2000]

Nonlinear AE is **strictly more powerful** than PCA — it can learn curved manifold structure.

Degenerate Solution Risk

If bottleneck is too wide ($L \geq D$), model can learn the identity. Must **restrict capacity**.

Bottleneck Auto-encoders.

Bottleneck the size of the code $r(\mathbf{x})$ to be L where $L \ll D$. Narrow bottleneck forces the model to learn a compressed representation.

- Simple, effective
- Prevents identity mapping
- Risk: too small $\rightarrow$ poor quality

Example



Figure 20.17: Results of applying an autoencoder to the Fashion MNIST data. Top row are first 5 images from validation set. Bottom row are reconstructions. (a) MLP model (trained for 20 epochs). The encoder is an MLP with architecture 784-100-30. The decoder is the mirror image of this. (b) CNN model (trained for 5 epochs). The encoder is a CNN model with architecture Conv2D(16, 3×3 , same, selu), MaxPool2D(2x2), Conv2D(32, 3×3 , same, selu), MaxPool2D(2×2), Conv2D(64, 3×3 , same, selu), MaxPool2D(2×2). The decoder is the mirror image of this, using transposed convolution and without the max pooling layers. Adapted from Figure 17.4 of [Gér19]. Generated by `ae mnist tf.ipynb`.

Denoising Autoencoders (DAE)

Key idea: Corrupt the input, train to reconstruct the clean original.

$$\tilde{\mathbf{x}} \sim p_c(\tilde{\mathbf{x}}|\mathbf{x}), \quad \mathcal{L} = \|r(\tilde{\mathbf{x}}) - \mathbf{x}\|^2$$

Two corruption strategies:

1. **Gaussian noise:** $\tilde{\mathbf{x}} = \mathbf{x} + \varepsilon, \varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$
2. **Bernoulli dropout:** randomly zero out pixels

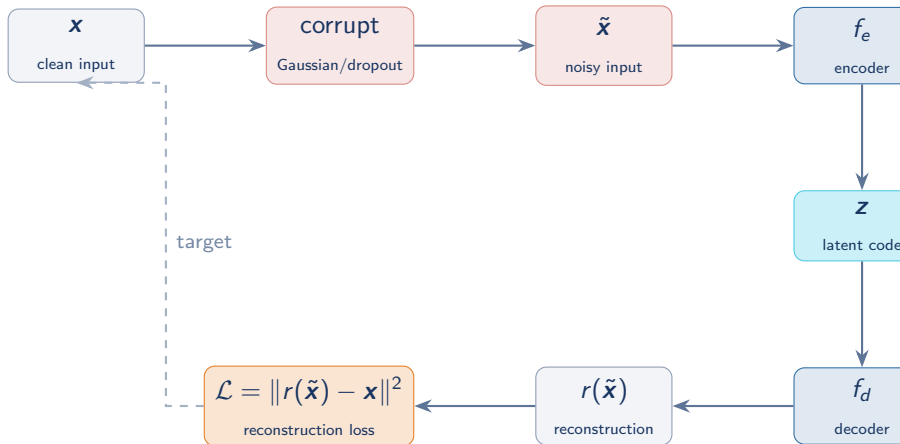
Remarkable Theorem [Alain & Bengio, 2014]

As $\sigma \rightarrow 0$, the DAE residual $\mathbf{e}(\mathbf{x}) = r(\tilde{\mathbf{x}}) - \mathbf{x}$ approximates the **score function**:

$$\mathbf{e}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

The DAE learns a **vector field** pointing toward the data manifold!

Continued



Sparse Autoencoders

Idea: Encourage most latent units to be inactive at any given time.

Method 1 — ℓ_1 activity regularization:

$$\mathcal{L}(\theta) = \|r(\mathbf{x}) - \mathbf{x}\|^2 + \lambda \|\mathbf{z}\|_1$$

Method 2 — KL sparsity [Glorot et al., 2011]:

$$\Omega = \lambda \sum_k D_{KL}(p\|z_k), \quad D_{KL}(p\|z_k) = p \log(p/z_k) + (1-p) \log \frac{1-p}{1-z_k}$$

where p is a desired activation rate (e.g., 0.1) and z_k is the empirical rate of unit k . The $z_k \in [0, 1]$ values are typically obtained via a last sigmoid unit in the encoder f_e .

The second method leads to each dimension being sparse p fraction of times because the KL divergence is applied individually on each dimension. In contrast the first method can cause many coordinates to be always off (not desired).

Summary

Variant	How capacity is controlled	Key property	Cost
Bottleneck AE	Narrow $\mathbf{z}$ ($L \ll D$)	Simple, general	Low
Denoising AE	Corrupt input $\tilde{\mathbf{x}}$	Learns score fn	Low
Sparse AE	ℓ_1 or KL on $\mathbf{z}$	Interpretable units	Low

Key Concepts

- Linear AE $\equiv$ PCA; nonlinear AE is strictly more powerful
- Must restrict capacity to avoid identity mapping
- **Reconstruction loss** drives both encoder and decoder
- DAE learns the **score function** — connection to diffusion models